DataQualityAnalysis

November 24, 2025

0.0.1 DB connection

[1]:
```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine

import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import gaussian_kde

import pandas as pd
import numpy as np

import numpy as np

load_dotenv()

DB_USER = os.getenv("DB_USER")
DB_PASS = os.getenv("DB_PASS")
DB_NAME = os.getenv("DB_NAME")
DB_HOST = os.getenv("DB_HOST")
DB_PORT = os.getenv("DB_PORT")

engine = create_engine(
    f"postgresql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
)

df_sales = pd.read_sql("SELECT * FROM sale_offers;", engine)
df_rentals = pd.read_sql("SELECT * FROM rental_offers;", engine)
df_sale_owners = pd.read_sql("SELECT * FROM sale_owners;", engine)
df_rental_owners = pd.read_sql("SELECT * FROM rental_owners;", engine)
```

1 Data Quality Analysis

1.1 Introduction

1.1.1 What is good quality data?

Good quality data means data that is fit for use. It is measured by: 1. Completeness: Nothing important is missing. 2. Uniqueness: No duplicates. 3. Validity: Follows rules, formats and constraints. 4. Accuracy: Correct and reflects reality. 5. Consistency: Relationships between data are correct. 6. Integrity: Matches across systems or tables.

1.1.2 What is my goal?

Create an automated ETL process that takes data from staging to production where it will be used for dashboard visualization and machine learning. The market in study is the housing market in the Metropolitan Area of the Aburrá Valley.

1.1.3 What will be done?

I'll review each criteria for the given dataset to identify where it needs cleaning and define how the ETL process should work.

1.1.4 First look

I'll perform a basic descriptive analysis for the dataset. Then, I'll create graphs that show some column's distributions.

[2]:
```
df_sales.head(5)
```

[2]:
```
   property_id   owner_id operation_type property_type department      city  \
0    193103868  174690599          Venta   Apartamento  Antioquia  Medellín
1    193103604  174787115          Venta   Apartamento  Antioquia  Medellín
2    193104035  176485821          Venta          Casa  Antioquia  Medellín
3    193104381  176480712          Venta   Apartamento  Antioquia  Medellín
4    193103855  174690599          Venta   Apartamento  Antioquia  Medellín

  main_location_name main_location_type  area_sqm  built_area_sqm  …  \
0          Castropol      neighbourhood     97.27           97.27  …
1          San diego      neighbourhood     57.00           57.00  …
2            Robledo      neighbourhood    160.00          160.00  …
3              Belén      neighbourhood     76.83           76.83  …
4            La mota      neighbourhood     80.00           80.00  …

   bathrooms  parking_spaces  penthouse  \
0          2               2      False
1          2               2      False
2          3               0      False
3          2               1      False
4          2               1      False
```

```
                                        facilities  \
0  Alarma; Ascensor; Balcón; Barra estilo america…
1  Acceso Pavimentado; Asador; Ascensor; Balcón; …
2
3  Calentador; Cocina Integral; Garaje(s); Hall d…
4  Alarma; Ascensor; Balcón; Barra estilo america…

                              close_neighbourhoods  offer_create_date  \
0                          Castropol; La candelaria         2025-11-16
1  San diego; Loma del Indio; Asomadera no.2; Aso…         2025-11-16
2  Cucaracho; Urbanizacion manesa; Palenque; Robledo         2025-11-16
3                   Rodeo Alto; Belén; Belén Rincón         2025-11-16
4                             La mota; La candelaria         2025-11-16

   offer_update_date                                        title  \
0         2025-11-16  Apartamento en Venta en Castropol, Medellín
1         2025-11-16  Apartamento en Venta en San diego, Medellín
2         2025-11-16           Casa en Venta en Robledo, Medellín
3         2025-11-16      Apartamento en Venta en Belén, Medellín
4         2025-11-16    Apartamento en Venta en La mota, Medellín

                                              url      price
0  /apartamento-en-venta-en-castropol-medellin/19…  680000000
1  /apartamento-en-venta-en-san-diego-medellin/19…  530000000
2       /casa-en-venta-en-robledo-medellin/193104035  470000000
3  /apartamento-en-venta-en-belen-medellin/193104381  289900000
4  /apartamento-en-venta-en-la-mota-medellin/1931…  650000000

[5 rows x 25 columns]
```

[3]:
```
df_rentals.head(5)
```

[3]:
```
   property_id   owner_id operation_type property_type department      city  \
0    192522216  174676173       Arriendo   Apartamento  Antioquia  Medellín
1    192525538  174647123       Arriendo   Apartamento  Antioquia  Medellín
2    192526300  174845933       Arriendo          Casa  Antioquia  Medellín
3    192526165  174845933       Arriendo          Casa  Antioquia  Medellín
4    192522569  174585359       Arriendo   Apartamento  Antioquia  Medellín

  main_location_name main_location_type  area_sqm  built_area_sqm  …  \
0         El Poblado      neighbourhood     78.00           78.00  …
1             Oviedo      neighbourhood     80.00             NaN  …
2          El tesoro      neighbourhood    250.00          250.00  …
3           Laureles      neighbourhood    341.25          341.25  …
4             Zuñiga      neighbourhood    120.00             NaN  …

   bathrooms  parking_spaces  penthouse  \
```

```
0          2               2      False
1          2               1      False
2          4               3      False
3          4               1      False
4          2               1      False

                                        facilities  \
0
1  Acceso Pavimentado; Ascensor; Balcón; Baño Aux…
2  Calefacción Individual; Alarma; Amoblado; Asce…
3  Ascensor(es) inteligente(s); Bahía exterior de…
4  Acceso Pavimentado; Ascensor; Balcón; Baño Aux…

                              close_neighbourhoods  offer_create_date  \
0                 Simesa; El Poblado; Ciudad del Río         2025-06-12
1      La aguacatala; Oviedo; Los Balsos; El Poblado         2025-06-13
2  El tesoro; Los naranjos; Transversal Superior;…         2025-06-13
3  Laureles; Belén Rosales; Belén; Laureles Nogal…         2025-06-13
4  Santa maria de los angeles; Bosques de zuñiga;…         2025-06-12

   offer_update_date                                          title  \
0         2025-07-11  Apartamento en Arriendo en El Poblado, Medellín
1         2025-06-17      Apartamento en Arriendo en Oviedo, Medellín
2         2025-11-14          Casa en Arriendo en El tesoro, Medellín
3         2025-11-14           Casa en Arriendo en Laureles, Medellín
4         2025-06-12      Apartamento en Arriendo en Zuñiga, Medellín

                                              url monthly_fee
0  /apartamento-en-arriendo-en-el-poblado-medelli…     7500000
1  /apartamento-en-arriendo-en-oviedo-medellin/19…     4300000
2  /casa-en-arriendo-en-el-tesoro-medellin/192526300    11213000
3   /casa-en-arriendo-en-laureles-medellin/192526165     8700000
4  /apartamento-en-arriendo-en-zuñiga-medellin/19…     4500000

[5 rows x 25 columns]
```

Both tables have 25 columns, the only difference being the column “price” which in rental_offers is named “monthly_fee”.

“main_location_type” affects the meaning of the values in the “main_location_name” column.

Each offer has an “owner_id” field which is the offer’s owner ID on the webpage, more information about each owner can be found in tables “sale_owners” and “rental_owners” depending on the offer type.

Descriptive Analysis

[4]:
```
df_sales.describe()
```

[4]:

	property_id	owner_id	area_sqm	built_area_sqm
count	1.965700e+04	1.965700e+04	19596.000000	15996.000000
mean	1.809378e+08	1.748635e+08	179.727671	178.167478
std	4.469467e+07	4.304801e+05	1181.527976	753.658298
min	1.127158e+06	1.745272e+08	0.000000	0.000000
25%	1.922784e+08	1.746324e+08	70.000000	70.000000
50%	1.926662e+08	1.747064e+08	96.000000	99.000000
75%	1.929170e+08	1.748292e+08	161.000000	166.000000
max	1.931062e+08	1.764858e+08	133377.000000	55447.000000

	strata	latitude	longitude	age	bedrooms
count	19657.000000	19657.000000	19657.000000	19306.000000	19657.000000
mean	4.574604	6.199839	-75.540563	2.532218	3.035763
std	5.878929	0.165892	1.867420	1.460850	1.071038
min	0.000000	0.000000	-76.241675	0.000000	-1.000000
25%	4.000000	6.160939	-75.606551	2.000000	3.000000
50%	4.000000	6.188054	-75.583696	2.000000	3.000000
75%	5.000000	6.239565	-75.566787	4.000000	3.000000
max	200.000000	9.498611	0.000086	5.000000	37.000000

	bathrooms	parking_spaces	price
count	19657.00000	19657.000000	1.965700e+04
mean	2.70031	1.406115	1.275342e+09
std	1.23627	1.533027	1.701989e+10
min	0.00000	-2.000000	0.000000e+00
25%	2.00000	1.000000	4.300000e+08
50%	2.00000	1.000000	6.500000e+08
75%	3.00000	2.000000	1.100000e+09
max	18.00000	84.000000	1.590000e+12

Note: Analysis may be unreliable until proper cleaning is done.

df_sales describe analysis:

- ☒ area_sqm:
  - ~179.73 m^2 is the average area of a property listed for sale.
  - The standard deviation is bigger than the mean by orders of magnitude, this suggests a non-homogeneous sample.
  - The minimum area is 0 m^2 which is unacurrate.
  - The median is 96 m^2, almost half the mean which suggests the data may be heavily skewed to the right.
  - The maximum is ~1.3e5 m^2, this seems unaccurate for a property in the metropolitan area.
- ☒ built_area_sqm:
  - It appears less than area_sqm.
  - ~178.17 m^2 is its average, less than the average of area_sqm which seems consistent.
  - Its standard deviation is ~753 m^2, this is less than the standard deviation of "area_sqm" column. This may be related to the fact that properties must meet certain built-up area limits in order to be approved.

- Similar analysis to "area_sqm" applies to its minimum and maximum values.
- The median is 99 m^2 which suggests the "built_area_sqm" column may be significantly skewed to the right too.

☒ strata:
- strata is a categorical column that should range between 1 and 6 according to *Law 142 of 1992*. It is not the case in the data, the minimum is 0 and the maximum is 200.
- 4 appears to be the median stratum in the sample.

☒ latitude:
- The Aburrá Valley's latitudes range from 6 North to 6.5 North, that is why a minimum of 0 and a maximum of ~9.5 are unaccurate.
- The mean and the median are inside the given range.

☒ longitude:
- The Aburra Valley's longitudes range from 75.3 West and 75.6 West, West coordinates are represented as negative longitudes. Given this context, mean and median are inside the given range.
- The minimum and maximum longitudes are not inside this range and are therefore unaccurate.

☒ age:
- Age is a categorical column that should range from 1 to 5. However, the minimum is 0 which may indicate an unaccurate value or a null placeholder.
- 2 is the median (1 to 8 years), this suggests houses for sale tend to be less than 8 years old.

☒ bedrooms:
- The mean and median are close together, this suggests a symetrical distribution, However, 3 seems to be the most common value in houses for sale.
- The minimum is a negative number, this is unacurrate.
- The maximum is 37 which seems excesive, this may be unacurrate or unconsistent with the listed offer (maybe a building for sale).

☒ bathrooms:
- Most houses for sale have between 2 and 3 bathrooms.
- The maximum value (18) may be unacurrate.

☒ parking_spaces:
- Most houses have between 1 and 2 parking spaces.
- The minimum value is a negative number, this is unaccurate.
- The maximum is 84 which is unacurrate or unconsistent similar to "bedrooms" analysis.

☒ **price**:
- The median price is 6.5e8 COP whilst the mean price is ~1.3e9 COP, the mean price is significantly bigger. This could indicate the price distribution is right skewed.
- The minimum price is unaccurate as there is a property listed for 0 COP.
- The maximum price is more than 1,000 times the median, seems unaccurate.

[5]:
```
df_rentals.describe()
```

[5]:

	property_id	owner_id	area_sqm	built_area_sqm	strata
count	1.549600e+04	1.549600e+04	1.540800e+04	1.263600e+04	15496.000000
mean	1.910983e+08	1.747609e+08	1.816298e+03	1.509731e+03	4.729801
std	1.766888e+07	2.228555e+05	9.223067e+04	6.723763e+04	6.135375

```
min    7.528422e+06  1.745281e+08  0.000000e+00    0.000000e+00      0.000000
25%    1.927215e+08  1.746643e+08  6.500000e+01    6.500000e+01      4.000000
50%    1.929146e+08  1.747108e+08  8.000000e+01    8.000000e+01      4.000000
75%    1.930245e+08  1.748292e+08  1.150000e+02    1.150000e+02      5.000000
max    1.931033e+08  1.764821e+08  8.600000e+06    5.500000e+06    110.000000

           latitude     longitude           age      bedrooms     bathrooms  \
count  15496.000000  15496.000000  14560.000000  15496.000000  15496.000000
mean       6.205249    -75.584519      2.261538      2.784138      2.331699
std        0.092407      0.608451      1.398734      1.071468      0.988383
min        0.000000    -76.663345      0.000000      0.000000      0.000000
25%        6.163672    -75.608763      1.000000      2.000000      2.000000
50%        6.201555    -75.589219      2.000000      3.000000      2.000000
75%        6.241461    -75.567698      3.000000      3.000000      3.000000
max        7.057580      0.000000      5.000000     25.000000     17.000000

       parking_spaces   monthly_fee
count    15496.000000  1.549600e+04
mean         1.039107  4.666094e+06
std          1.720936  1.851587e+07
min          0.000000  0.000000e+00
25%          0.000000  2.350000e+06
50%          1.000000  3.200000e+06
75%          1.000000  4.700000e+06
max        127.000000  1.670000e+09
```

df_rentals describe analysis:

Analysis checklist with annotations on differences between the previous analysis.

- ☒ area_sqm:
  - 80 m^2 is the median area of a rental property.
  - ~1,820 m^2 is the average area of a rental property, this is a heavy contrast to the median which suggests an outlier is present.
  - The maximum area is 8.6 million square meters which is greatly unaccurate.
- ☒ built_area_sqm:
- ☒ strata:
  - The minimum is 0 and the maximum is 110, both values are not inside the stipulated range.
- ☒ latitude
- ☒ longitude
- ☒ age
- ☒ bedrooms:
  - Minimum is 0 which is interesting for a property.
- ☒ bathrooms:
  - Most houses for rent in the sample appear to have between 2 and 3 bathrooms, tending to only two.
- ☒ parking_spaces:
  - Most houses for rent have only 1 parking space.

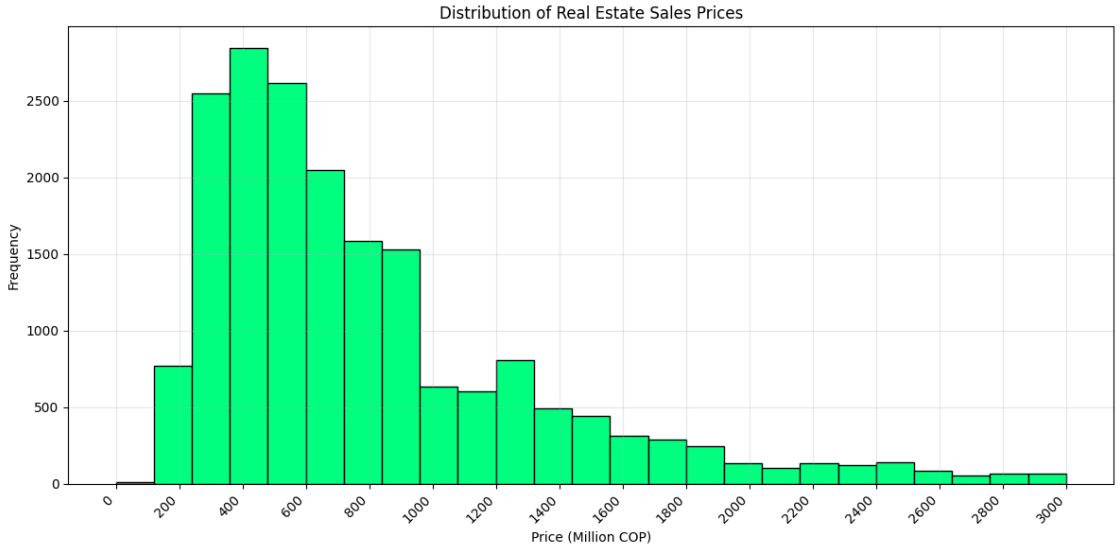
- The maximum is 127 which is unacurrate.

☒ **monthly_fee**:

- The median price is 3.2e6 COP whilst the mean price is ~4.7e6 COP, the sample appears to be slightly skewed (comparared to "price").
- The minimum price is unaccurate as there is a property listed for 0 COP.
- The maximum price is more than 100 times the median which is interesting.

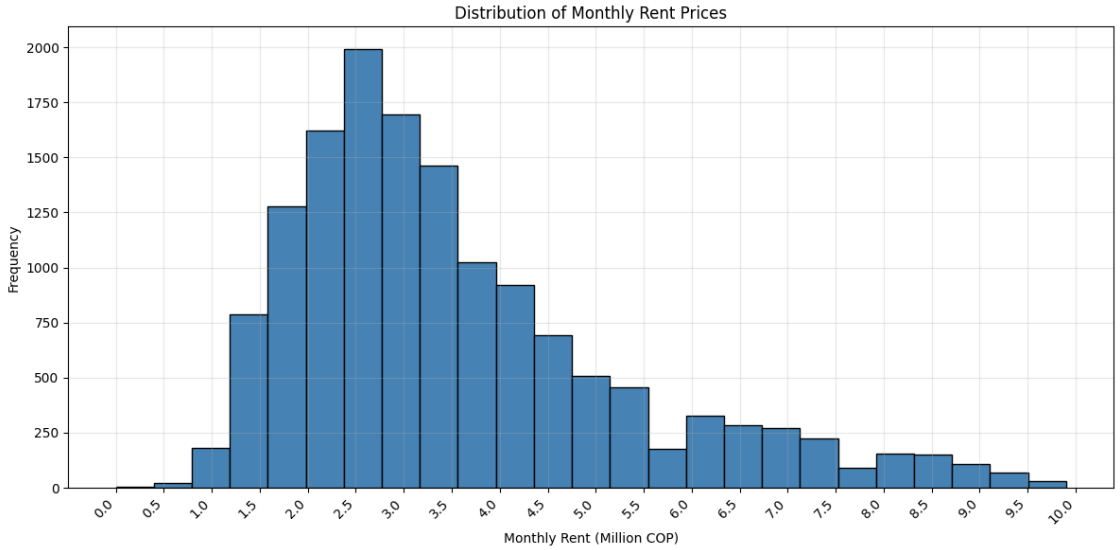
Graphs

[6]:
```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

prices_million = df_sales[df_sales["price"] < 3e9]["price"] / 1_000_000

fig, ax = plt.subplots(figsize=(12, 6))


prices_million.hist(bins=25, edgecolor="black", color="springgreen", ax=ax)

ax.set_title("Distribution of Real Estate Sales Prices")
ax.set_xlabel("Price (Million COP)")
ax.set_ylabel("Frequency")


# Grid
ax.yaxis.grid(True, alpha=0.3)
ax.xaxis.grid(True, alpha=0.3)

ax.ticklabel_format(style='plain', axis='x')
ax.xaxis.offsetText.set_visible(False)
ax.xaxis.set_major_locator(ticker.MultipleLocator(200))

plt.setp(ax.get_xticklabels(), rotation=45, ha="right")

plt.tight_layout()

#plt.savefig("distribution_of_real_estate_sales_prices.png")


plt.show()
```



[7]:
```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

rent_million = df_rentals[df_rentals["monthly_fee"] < 1e7]["monthly_fee"] /␣
 ↪1_000_000

fig, ax = plt.subplots(figsize=(12,6))

rent_million.hist(
    bins=25,
    edgecolor="black",
    color="steelblue",
    ax=ax
)

ax.set_title("Distribution of Monthly Rent Prices")
ax.set_xlabel("Monthly Rent (Million COP)")
ax.set_ylabel("Frequency")

ax.yaxis.grid(True, alpha=0.3)
ax.xaxis.grid(True, alpha=0.3)

ax.ticklabel_format(style='plain', axis='x')
ax.xaxis.offsetText.set_visible(False)

ax.xaxis.set_major_locator(ticker.MultipleLocator(0.5))

plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
```

```
plt.tight_layout()

plt.savefig("distribution_of_monthly_rent_prices.png")

plt.show()
```



1.2 Criteria Review

1.2.1 *df_sales*

Completeness: Nothing important is missing

[8]:
```
df_sales.isnull().sum()
```

[8]:
```
property_id              0
owner_id                 0
operation_type           0
property_type            0
department              12
city                     0
main_location_name      75
main_location_type      75
area_sqm                61
built_area_sqm        3661
strata                   0
latitude                 0
longitude                0
age                    351
bedrooms                 0
```

```
bathrooms                 0
parking_spaces            0
penthouse                 0
facilities                0
close_neighbourhoods      0
offer_create_date         0
offer_update_date         0
title                     0
url                       0
price                     0
dtype: int64
```

Only explicit null values appear on the following columns: - department - main_location_name - main_location_type - area_sqm - built_area_sqm - age

property_id Sorting by property_id doesn't show any common null placeholder.

This column seems complete.

[9]:
```
df_sales["property_id"].sort_values()
```

[9]:
```
15705      1127158
17219      4197365
13363      4368907
13364      4488985
13365      4661757
           ...
15       193105899
8953     193105901
8952     193106048
18       193106168
19       193106208
Name: property_id, Length: 19657, dtype: int64
```

owner_id First I started checking the most common values to check whether they may be null placeholders, for each I got the owner info.

[10]:
```
owner_counts = (
    df_sales.groupby("owner_id")["property_id"]
            .count()
            .rename("property_count")
)

most_common_owner_ids = (
    owner_counts.sort_values(ascending=False)
                .head(20)
                .index
)
```

```
filtered = df_sale_owners[df_sale_owners["owner_id"].
 ↪isin(most_common_owner_ids)]

filtered.merge(owner_counts, left_on="owner_id", right_index=True, how="left").
 ↪sort_values("property_count", ascending=False)
```

[10]:

	owner_id	owner_name	masked_phone	active	property_count
14	174681635	Prointegral	+5730	True	1157
35	175895178	CASAS Y ESPACIOS	+5731	True	1085
88	174706628	Grupo Empresarial Mi	+5732	True	762
59	174649964	RHBRAIZ	+5730	True	722
44	175459904	Tu360Inmobiliario en colaboración con Pulppo	+5733	True	681
87	174632350	BROKER	+5731	True	471
7	174626844	Viva Inmobiliaria Asociados	+5732	True	462
171	174716153	Solubienes Propiedad Raiz	+5760	True	441
25	174657612	Compro Casa	+5730	True	359
91	174673150	BEMSA	+5732	True	335
67	174548932	Mattis Inmobiliaria	+5732	True	329
183	174573641	HOMES PROVENTO	+5730	True	321
1	174739170	Inmobiliaria la Cumbre	+5732	True	293
163	174921490	Arrendamientos Envigado	31835	True	270
175	174625790	Murillo Propiedades	+5731	True	260
83	174578971	GRUPO INMOBILIARIO CABAL	+5731	True	222
31	174833234	RENTA RAÍZ SOLUCIONES INMOBILIARIAS SAS	+5735	True	222
45	174710825	Maxibienes S.A.S	+5731		
4	174845933	Engel & Völkers	+5760		
2	174529876	Century 21 Radial	+5731		

```
45      True              218
4       True              216
2       True              209
```

This column seems complete.

operation_type Every operation_type in this df is, as expected, "Venta" (sale). No null placeholders found.

This column seems complete.

[11]:
```
df_sales["operation_type"].unique()
```

[11]:
```
array(['Venta'], dtype=object)
```

Every name seems common, only one caught my attention which was "BROKER". This seemed like a generic name, investigating further it is a real state agency and not a placeholder.

This column seems complete.

department (12 null values) As seen in the general analysis, this column is not complete as it has 12 null entries.

There's no null placeholder.

[12]:
```
df_sales["department"].unique()
```

[12]:
```
array(['Antioquia', 'Valle del cauca', 'Bogotá, d.c.', 'Cundinamarca',
       None, 'Cesar'], dtype=object)
```

city There are no explicit null values.

There are no null placeholder.

This column seems complete.

[13]:
```
df_sales["city"].unique()
```

[13]:
```
array(['Medellín', 'Envigado', 'Itaguí', 'La estrella', 'Bello',
       'Sabaneta'], dtype=object)
```

main_location_name & main_location_type (75 null values) There are 75 explicit null values, a null in main_location_name is equivalent to a null in main_location_type.

[14]:
```
# main_location_type null <- main_location_name null

df_sales[df_sales["main_location_name"].isnull()]["main_location_type"].sum()
```

[14]:
```
0
```

[15]:
```
# main_location_type null -> main_location_name null

df_sales[df_sales["main_location_type"].isnull()]["main_location_name"].sum()
```

[15]:
```
0
```

[16]:
```
# No null value

df_sales.groupby("main_location_type").count()["property_id"]
```

[16]:
```
main_location_type
city              2792
commune            306
locality             1
neighbourhood    16373
zone               110
Name: property_id, dtype: int64
```

area_sqm (72 null values) There are 61 explicit null values.

0 is a null value with 11 occurrences

This sums up to 72 null values in the sample.

[17]:
```
df_sales["area_sqm"].sort_values()[:20].unique()
```

[17]:
```
array([0., 1.])
```

[18]:
```
(df_sales["area_sqm"] == 0).sum()
```

[18]:
```
np.int64(11)
```

built_area_sqm (3935 null values) There are 3661 explicit null values.

0 is a null value with 274 occurrences

This sums up to 3935 null values in the sample.

[19]:
```
df_sales["built_area_sqm"].sort_values()[:275].unique()
```

[19]:
```
array([0., 1.])
```

[20]:
```
(df_sales["built_area_sqm"] == 0).sum()
```

[20]:
```
np.int64(274)
```

strata (772 null values) Strata is a categorical variable, it should range from 1 to 6. However, the sample doesn't follow this pattern.

I focused on identifying the null placeholders.

0 and 110 are the null values identified, they appear on 772 rows.

[21]:
```
df_sales["strata"].unique()
```

[21]:
```
array([  5,   4,   3,   6,   0,   2, 110, 100,   1,   7, 200])
```

[22]:
```
df_sales[df_sales["strata"] == 0]["url"].iloc[0] # The webpage lists this␣
 ↪section as "ask the owner", no more info is provided.
#  It is a null placeholder.
```

[22]:
```
'/apartamento-en-venta-en-el-poblado-medellin/193060374'
```

[23]:
```
df_sales[df_sales["strata"] == 100]["url"].iloc[0] # "campestre" which is an␣
 ↪unofficial term to describe rural properties.
#  It is not a null placeholder.
```

[23]:
```
'/apartamento-en-venta-en-loma-del-indio-medellin/193060732'
```

[24]:
```
df_sales[df_sales["strata"] == 110]["url"].iloc[0] # The webpage lists this␣
 ↪section as undefined.
#  It is a null placeholder.
```

[24]:
```
'/apartamento-en-venta-en-el-poblado-medellin/193072527'
```

[25]:
```
df_sales[df_sales["strata"] == 7]["url"].iloc[0] # "campestre".
#  It is not a null placeholder.
```

[25]:
```
'/casa-en-venta-en-las-palmas-envigado/193013037'
```

[26]:
```
df_sales[df_sales["strata"] == 200]["url"].iloc[0] # Listed as 200, clearly an␣
 ↪error, but...
#  It is not a null placeholder.
```

[26]:
```
'/apartamento-en-venta-en-las-palmas-envigado/192650536'
```

[27]:
```
df_sales["strata"].isin([0, 110]).sum()
```

[27]:
```
np.int64(772)
```

latitude & longitude (12 null values) No explicit null values found.

Null placeholders are 0 and close to 0 values, these appear 12 times in the sample.

[28]:
```
df_sales[df_sales["latitude"] < 1] [["latitude", "longitude", "url"]].count()
```

[28]:
```
latitude     12
longitude    12
url          12
dtype: int64
```

age (3002 null values) 351 explicit null values as per general analysis.

0 is a null place holder which appears in 2651 entries

[29]:
```
df_sales["age"].unique()
```

[29]:
```
array([ 2.,  0.,  5.,  4.,  3., nan,  1.])
```

[30]:
```
(df_sales["age"] == 0).sum()
```

[30]:
```
np.int64(2651)
```

bedrooms (89 null values) No explicit null values as per general analysis.

0 is a null placeholder which appears in 89 entries

[31]:
```
df_sales[df_sales["bedrooms"] == 0]["url"].iloc[0] # bathroom number appears as␣
↪"ask the owner"
# It is a null placeholder.
```

[31]:
```
'/casa-en-venta-en-laureles-medellin/193100644'
```

[32]:
```
(df_sales["bedrooms"] == 0).sum()
```

[32]:
```
np.int64(89)
```

bathrooms (83 null values) No explicit null values as per general analysis.

0 is a null placeholder which appears in 83 entries

[33]:
```
df_sales[df_sales["bathrooms"] == 0]["url"].iloc[0] # bathroom number appears␣
↪as "ask the owner"
# It is a null placeholder.
```

[33]:
```
'/apartamento-en-venta-en-barrios-de-jesus-medellin/193104859'
```

[34]:
```
(df_sales["bathrooms"] == 0).sum()
```

[34]:
```
np.int64(83)
```

parking_spaces (null values) No explicit null values as per general analysis.

0 may or not be a placeholder (properties may have 0 parking_spaces), in the worst scenario there are 4208 null values.

[35]:
```
df_sales[df_sales["parking_spaces"] == 0]["url"].iloc[0] # parking_spaces␣
↪appears as "ask the owner"
# It is a null placeholder.
```

[35]:
```
'/casa-en-venta-en-robledo-medellin/193104035'
```

[36]:
```
(df_sales["parking_spaces"] == 0).sum()
```

[36]:
```
np.int64(4208)
```

penthouse (null values) No explicit null values as per general analysis.

False may or not be a placeholder, however, only 1 property for sale has been classified as a penthouse.

[37]:
```
df_sales["penthouse"].sum()
```

[37]:
```
np.int64(1)
```

facilities (1024 null values) No explicit null values as per general analysis.

The empty string '' is a null placeholder, there are 1024 entries with this value.

[38]:
```
(df_sales["facilities"] == '').sum()
```

[38]:
```
np.int64(1024)
```

close_neighbourhoods (64 null values) No explicit null values as per general analysis.

The empty string '' is a null placeholder, there are 64 entries with this value.

[39]:
```
(df_sales["close_neighbourhoods"] == '').sum()
```

[39]:
```
np.int64(64)
```

offer_create_date No explicit null values as per general analysis.

No null place holders found.

This column seems complete.

df_sales.sort_values("offer_create_date")["offer_create_date"]

offer_update_date No explicit null values as per general analysis.

No null place holders found.

This column seems complete.

[40]:
```
df_sales.sort_values("offer_update_date")["offer_update_date"]
```

[40]:
```
16432    2025-03-14
11777    2025-03-17
8920     2025-03-18
8924     2025-03-18
16427    2025-03-18
            ...
```

```
2891     2025-11-18
5128     2025-11-18
16892    2025-11-18
15853    2025-11-18
8256     2025-11-18
Name: offer_update_date, Length: 19657, dtype: object
```

title No explicit null values as per general analysis.

No null place holders found.

This column seems complete.

url No explicit null values as per general analysis.

No null place holders found.

This column seems complete.

price No explicit null values as per general analysis.

0 is a value and it is related to a precision error, not a completeness issue.

[41]:
```
df_sales["price"].sort_values()[:50].unique()
```

[41]:
```
array([        0,         1,   1200000,   1290000,   1550000,   2962400,
         3000000,   7000000,   7500000,   8300000,  12000000, 102000000,
       115000000, 120000000, 125000000, 128000000, 130000000, 131300000,
       135000000, 140000000, 142000000, 142500000, 145000000, 146000000,
       148900000, 149000000, 150000000])
```

[42]:
```
(df_sales["price"] == 0).sum()
```

[42]:
```
np.int64(2)
```

df_rentals A similar analysis was performed on the *df_rentals* table giving the same results as *df_sales*.

Remember, the main concern is how to create the automated ETL process from staging DB to the production DB, therefore, knowing which columns are affected by null values is the priority.

Results (null affected columns):

1. department
2. main_location_name
3. main_location_type
4. area_sqm
5. built_area_sqm
6. strata
7. latitude
8. longitude

9. age
10. bedrooms
11. bathrooms
12. parking_spaces
13. penthouse
14. facilities
15. close_neighbourhoods

Columns marked with are those with an uncertain number of nulls.

This columns aforementioned must receive special treatment to be of use in the modelling phase, they will be taken into account in the automated ETL process I'll create later on.

[43]:
```
#### Dumb cleaning to focus on the next section...

df_sales.loc[df_sales["area_sqm"] == 0, "area_sqm"] = np.nan

df_sales.loc[df_sales["built_area_sqm"] == 0, "built_area_sqm"] = np.nan

df_sales.loc[df_sales["strata"] == 0, "strata"] = np.nan
df_sales.loc[df_sales["strata"] == 110, "strata"] = np.nan

df_sales.loc[(df_sales["latitude"] < 1), "latitude"] = np.nan

df_sales.loc[df_sales["age"] == 0, "age"] = np.nan

df_sales.loc[df_sales["bedrooms"] == 0, "bedrooms"] = np.nan

df_sales.loc[df_sales["bathrooms"] == 0, "bathrooms"] = np.nan

df_sales.loc[df_sales["facilities"] == '', "facilities"] = np.nan

df_sales.loc[df_sales["close_neighbourhoods"] == '', 'close_neighbourhoods'] =␣
 ↪np.nan



df_sales.dropna(subset=['area_sqm', 'price', 'bedrooms','latitude',␣
 ↪'longitude', 'main_location_type'], inplace=True)
```

[ ]:
```

```

Uniqueness: No duplicates Due to the way the data was collected, no complete duplicates are possible since property_id was defined to be unique in the database.

However, the same property may be offered by others, I'll create a way to detect this type of duplicates.

Two properties will be flagged as the same if they share their.. - area_sqm value - property_type

value - department value - city value - number of bedrooms - close_neighbourhoods values - price value

Note: Offering a property has a cost of 51,600 COP (13,72 USD)

Duplicate example: - https://www.fincaraiz.com.co/apartamento-en-venta-en-santa-ana-bello/193042690 [First Photo] - https://www.fincaraiz.com.co/apartamento-en-venta-en-santa-ana-bello/192963170 [Sixth Photo]

[44]:
```
cols = ["property_type", "department", "city", "area_sqm", "bedrooms",␣
 ↪"close_neighbourhoods", "price"]

df_sales_duplicates = df_sales.copy()

df_sales_duplicates["duplicate_group"] = (
    df_sales.groupby(cols, dropna=False)
      .ngroup() + 1
)

group_counts = df_sales_duplicates["duplicate_group"].value_counts()
valid_groups = group_counts[group_counts > 1].index

df_sales_duplicates =␣
 ↪df_sales_duplicates[df_sales_duplicates["duplicate_group"].
 ↪isin(valid_groups)]

df_sales_duplicates.sort_values("duplicate_group", inplace=True)
```

[45]:
```
len(df_sales_duplicates)
```

[45]:
```
1108
```

[46]:
```
str((len(df_sales_duplicates) / len(df_sales)) * 100) + '%'
```

[46]:
```
'5.707809602307851%'
```

Results: The dataset has duplicates: same property listed by different agencies or the same.

In the worst case scenario, it aproximately 1115 duplicates which would account for the **5.7%** of the dataset.

Later on, I'll use p-hashes to identify duplicates using website images.

df_rentals: A similar analysis on *df_rentals* was done. There are duplicates too. Same approach will be used.

Validity: Follows rules, formats and constraints. There are some rules the data must follow to be considered valid. For each column… - id's: must be a positive integer, *property_id* must be unique.

- operation_type: "Venta" (sale) or "Arriendo" (rental).

- property_type: “Casa” (house) or “Apartamento” (apartment).
- department: “Antioquia”.
- city: Must belong to the set of scrapped cities.
- main_location_type: belong to the set of location types.
- area_sqm: Non-negative float, cannot be null.
- built_area_sqm: Non-negative float, lesser or equal than area_sqm, may be null.
- strata: must belong to {1, 2, 3, 4, 5, 6, nan}.
- latitude: within [-90, 90].
- longitude: within [-180, 180].
- age: Must belong to {1, 2, 3, 4, 5, nan}
- bedrooms: positive integer.
- bathrooms: positive integer.
- parking_spaces: non-negative float.
- penthouse: boolean.
- facilities: strings or empty strings.
- close_neighbourhoods: strings.
- offer_create_date: yyyy-mm-dd valid format.
- offer_update_date: newer than offer_create date, same format.
- title: string.
- price: positive.

Results

[47]:
```
# Checking all columns...

if (df_sales["property_id"] > 0).all and (df_sales["property_id"].is_unique):␣
 ↪print("Valid property_id")

if (df_sales["owner_id"] <= 0).any(): print("0 or negative.")
else: print("Valid owner_id.")

if(df_sales["operation_type"].unique() == ["Venta"]): print("Valid␣
 ↪operation_type.")
if(df_sales["property_type"].isin(["Apartamento", "Casa"]).all()): print("Valid␣
 ↪property_type.")
if (df_sales["department"] == "Antioquia").all(): print("Valid department")
else: print("  Invalid department.")

cities=["Medellín", "Envigado", "Itaguí", "La estrella", "Sabaneta", "Bello"]


if(df_sales["city"].isin(cities).all()): print("Valid city.")
else: print("  Invalid city.")


valid_location_types = ["neighbourhood", "city", "zone", "commune", "locality",␣
 ↪np.nan]
```

```
if (df_sales["main_location_type"].isin(valid_location_types)).all():
 ↪print("Valid location type.")
else: print("  Invalid main_location_type")

if (df_sales["area_sqm"] >= 0).all(): print("Valid area_sqm.")
else: print("  Invalid area_sqm.")

if (df_sales["built_area_sqm"] >= 0).all(): print("Valid area_sqm.")
else: print("  Invalid built_area_sqm.")

valid_strata = [1, 2, 3, 4, 5, 6, np.nan]

if df_sales["strata"].isin(valid_strata).all(): print("Valid strata.")
else: print("  Invalid strata.")

if ((df_sales["latitude"]>= (-90)) & (df_sales["latitude"] <= 90)).all():␣
 ↪print("Valid latitude.")
else: print("  Invalid latitude.")

if ((df_sales["longitude"]>= (-180)) & (df_sales["latitude"] <= 180)).all():␣
 ↪print("Valid longitude.")
else: print("  Invalid longitude.")

valid_age = [1, 2, 3, 4, 5, np.nan]

if df_sales["age"].isin(valid_age).all(): print("Valid age.")
else: print("  Invalid age.")

if (df_sales["bedrooms"] > 0).all() and df_sales["bathrooms"].apply(lambda x:␣
 ↪isinstance(x, int)).all(): print("Valid bedrooms.")
else: print("  Invalid bedrooms.")

if ((df_sales["bathrooms"] > 0).all() and df_sales["bathrooms"].apply(lambda x:␣
 ↪isinstance(x, int)).all()): print("Valid bathrooms.")
else:
    print("  Invalid bathrooms.")

if (df_sales["parking_spaces"] >= 0).all() and df_sales["parking_spaces"].
 ↪apply(lambda x: isinstance(x, int)).all():
    print("Valid parking_spaces.")
else:
    print("  Invalid parking_spaces.")

if df_sales["penthouse"].apply(lambda x: isinstance(x, bool)).all():␣
 ↪print("Valid penthouse.")
else:
```

```
    print("  Invalid penthouse.")

if df_sales["close_neighbourhoods"].apply(lambda x: (isinstance(x, str) or np.
 ↪isnan(x))).all(): print("Valid close_neighbourhoods.")
else:
    print("  Invalid close_neighbourhoods.")

cast_create_date = pd.to_datetime(df_sales["offer_create_date"],␣
 ↪format="%Y-%m-%d", errors="coerce")


if cast_create_date.isnull().any():
    print("  Invalid offer_create_date.")
else:
    print(" Valid offer_create_date.")


cast_update_date = pd.to_datetime(df_sales["offer_update_date"],␣
 ↪format="%Y-%m-%d", errors="coerce")

if cast_update_date.isnull().any() or (cast_update_date < cast_create_date).
 ↪any():
    print("  Invalid offer_update_date.")
else:
    print(" Valid offer_update_date.")

if df_sales["title"].apply(lambda x: isinstance(x, str)).all(): print("Valid␣
 ↪close_neighbourhoods.")

if (df_sales["price"] > 0).all(): print("Valid price.")
else: print("  Invalid price")
```

```
Valid property_id
Valid owner_id.
Valid operation_type.
Valid property_type.
  Invalid department.
Valid city.
Valid location type.
Valid area_sqm.
  Invalid built_area_sqm.
  Invalid strata.
Valid latitude.
Valid longitude.
Valid age.
  Invalid bedrooms.
  Invalid bathrooms.
```

```
 Invalid parking_spaces.
Valid penthouse.
Valid close_neighbourhoods.
 Valid offer_create_date.
 Valid offer_update_date.
Valid close_neighbourhoods.
 Invalid price
```

[48]:
```python
#### Dumb cleaning to focus on the next section...

#  Invalid department.
df_sales = df_sales[df_sales["department"] == "Antioquia"]

#  Invalid built_area_sqm.
df_sales = df_sales[df_sales["area_sqm"] >= df_sales["built_area_sqm"]]

#  Invalid strata.
df_sales.loc[df_sales["strata"] == 7, "strata"] = 6 # "Campestre" which is␣
 ↪usually understood as a "Large property".
df_sales.loc[df_sales["strata"] == 100, "strata"] = 6 # "Campestre"
df_sales.loc[df_sales["strata"] == 200, "strata"] = 6 # One property identified␣
 ↪as this strata.

#  Invalid bedrooms.
df_sales = df_sales[df_sales["bedrooms"] > 0]

#  Invalid bathrooms.
df_sales = df_sales[df_sales["bathrooms"] > 0]

# Invalid parking_spaces.
df_sales = df_sales[df_sales["parking_spaces"] > 0]

# Invalid price
df_sales = df_sales[df_sales["price"] > 0]
```

Accuracy: Correct and reflects reality. Each column must be correct and reflect reality, this is defined for each relevant column as follows:

- main_location_name: must match the type given by main_location_type.
- area_sqm: must be within 30 m^2 and 10,000 m^2.
- built_area_sqm:
  - In many cases built_area_sqm is said to be equal to area_sqm but this is not plausible for big properties. The maximum built area of a real property I could find was 2000 m^2, this is exceeded by some offers.
- strata: same as validity check.

- latitude: must belong to **[6, 6.5]** as stipulated in the describe analysis.
- longitude: similarly to latitude, must belong to **[-75.6, -75.3]**.
- age: same as validity check.
- bedrooms: lesser or equal than 10. (less than 0.5%)
- bathrooms: lesser or equal than 10. (less than 0.5%)
- parking_spaces: lesser or equal than 10. (less than 0.5%)
- facilities:
  - non contradicting facilities for the same property ('Rural Area' and 'Urban Area', or, 'In Mall' and 'Residential Area')
  - no commercial facilities that indicate the property is neither a house or an apartment but an office or commercial site.
- close_neighbourhoods: (needs API integration) close_neighbourhoods must be close to the location of the property, for this, the Overpass API may be used.
- offer_create_date: not before the website, not after the scraping date.
- offer_update_date: same as offer_create_date.
- **price**: must be within 50 million COP and 50 billion COP.

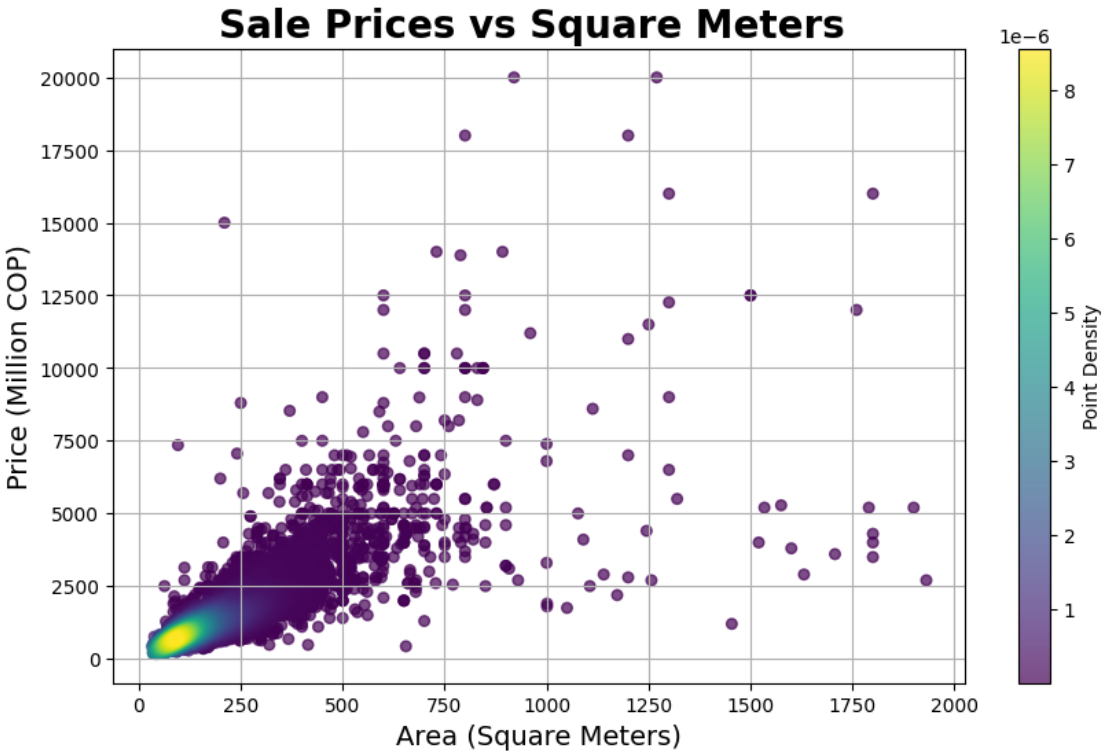
[49]:
```
#### Dumb cleaning to focus on the next section...

df_sales = df_sales[(df_sales["area_sqm"] > 30) & (df_sales["area_sqm"] <␣
↪10000)]

df_sales = df_sales[(df_sales["built_area_sqm"] > 30) &␣
↪(df_sales["built_area_sqm"] < 2000)]

df_sales = df_sales[(df_sales["latitude"] > 6) & (df_sales["latitude"] < 6.5)]
df_sales = df_sales[(df_sales["longitude"] > (-75.6)) & (df_sales["longitude"]␣
↪< (-75.3))]

df_sales = df_sales[(df_sales["bedrooms"] < 10)]
df_sales = df_sales[(df_sales["bathrooms"] < 10)]
df_sales = df_sales[(df_sales["parking_spaces"] < 10)]

# "Facility" validity check will be done before the modelling phase.

df_sales = df_sales[(df_sales["price"] >= 50_000_000) & (df_sales["price"] <=␣
↪50_000_000_000)]
```

Consistency: Relationships between data are correct. I will compare multiple columns to check for inconsistencies. - area_sqm & price - area_sqm & property_type - area_sqm & bathrooms

The easiest way to know what to look for is to graph the distribution of the studied variables so as to know what is “normal”.

Graphs area_qm & price

[50]:
```
# Extract data
x = df_sales['area_sqm']
y = df_sales['price'] / 1_000_000

# Calculate point density
xy = np.vstack([x, y])
z = gaussian_kde(xy)(xy)

# Sort points by density so dense points are plotted on top
idx = z.argsort()
x, y, z = x.iloc[idx], y.iloc[idx], z[idx]

plt.figure(figsize=(10,6))
plt.scatter(x, y, c=z, s=30, cmap='viridis', alpha=0.7)
plt.colorbar(label='Point Density')

plt.title('Sale Prices vs Square Meters ', fontsize=20, weight='bold')
plt.xlabel('Area (Square Meters)', fontsize=14)
plt.ylabel('Price (Million COP)', fontsize=14)


plt.grid(True)
plt.show()
```

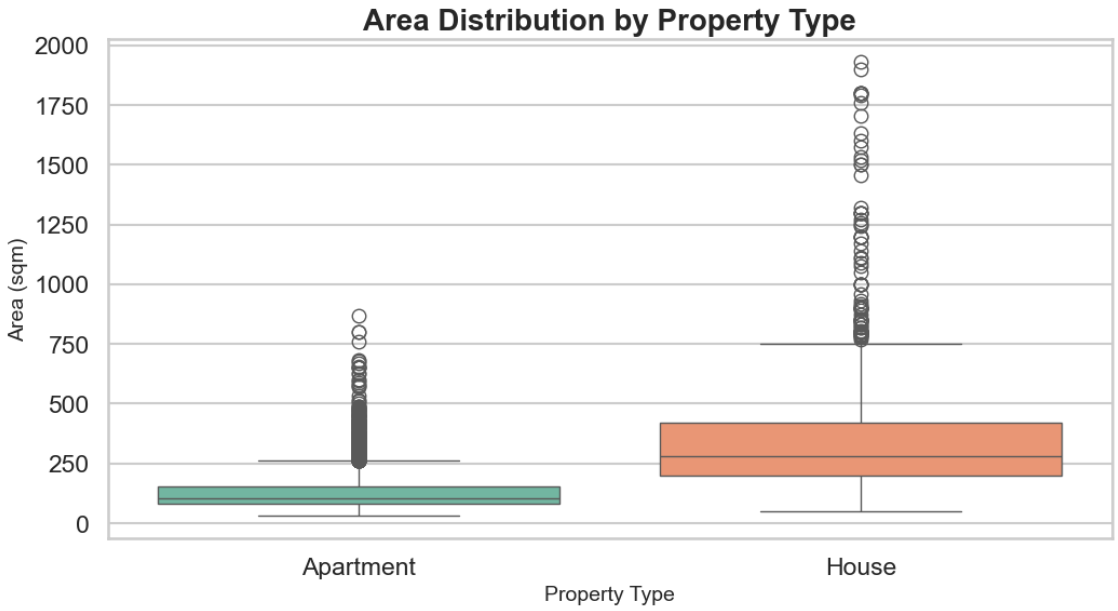


It is fair to say there are inconsistent entries in the before seen graph. A statistical analysis must be conducted to pick the best method to detect said anomalies. This will be done in the "Anomaly Detection" notebook.

area_sqm & property_type

[51]:
```
# Label mapping for visualization only
label_map = {
    'Apartamento': 'Apartment',
    'Casa': 'House'
}

plt.figure(figsize=(12,6))
sns.set_style("whitegrid")
sns.set_context("talk")

# Map x labels for hue without changing the df
sns.boxplot(
    x=df_sales['property_type'].map(label_map),  # mapped labels
    y=df_sales['area_sqm'],
    hue=df_sales['property_type'].map(label_map),  # hue now matches x
    palette='Set2',
    dodge=False  # prevent duplicate boxes
```

```
)

plt.title('Area Distribution by Property Type', fontsize=20, weight='bold')
plt.xlabel('Property Type', fontsize=14)
plt.ylabel('Area (sqm)', fontsize=14)
plt.xticks(rotation=0)
plt.show()
```



As anticipated, apartments are generally smaller than houses. In both property types, the distributions exhibit a "long-tail" pattern, with the median value falling below the mean.

Using 3*IQR for outlier detection, using only these two variables it can be seen that there are, for each property type,

[52]:
```
property_types = df_sales['property_type'].unique()

for prop in property_types:
    values = df_sales.loc[df_sales['property_type'] == prop, 'area_sqm']

    q1 = np.percentile(values, 25)
    q3 = np.percentile(values, 75)
    iqr = q3 - q1

    lower_fence = q1 - 3 * iqr
    upper_fence = q3 + 3 * iqr

    outliers = values[(values < lower_fence) | (values > upper_fence)]
```

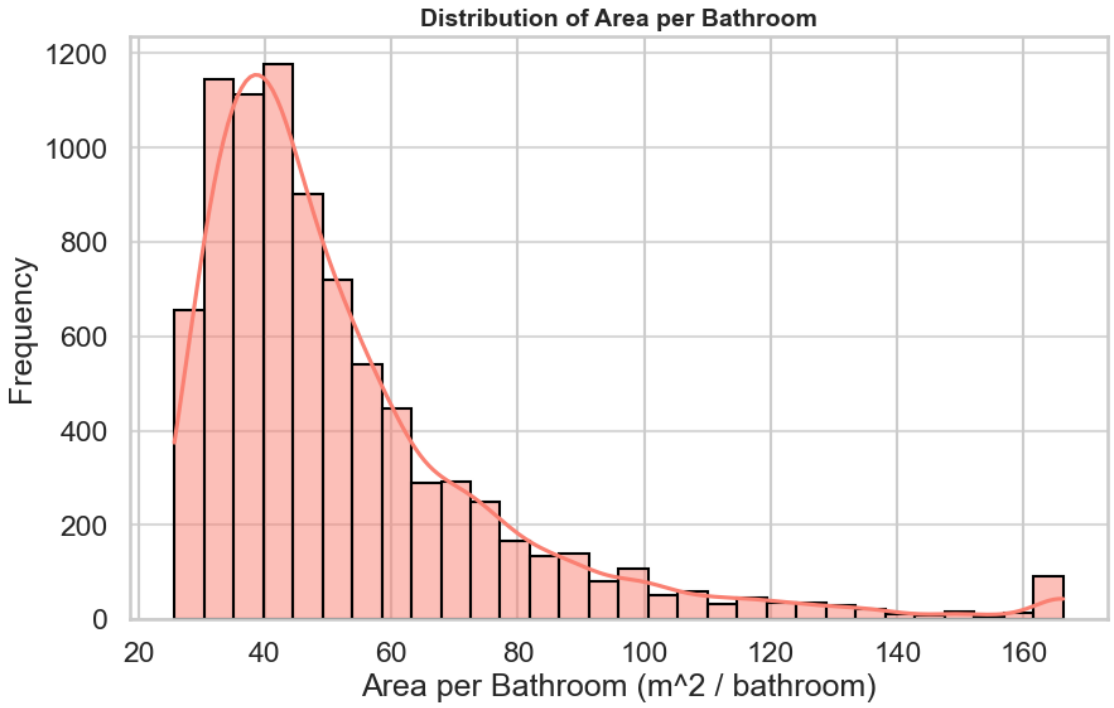
```
    print(f"'{prop}': {len(outliers)} outliers out of {len(values)} properties␣
↪({round((len(outliers) / len(values)) * 100, 2)}%)")
```

```
'Apartamento': 97 outliers out of 6610 properties (1.47%)
'Casa': 35 outliers out of 2000 properties (1.75%)
```

area_sqm vs bathrooms:

First I graphed the distribution of the relation between bathrooms and area_sqm.

[53]:
```
area_per_bathroom = df_sales['area_sqm'] / df_sales['bathrooms']

lower = area_per_bathroom.quantile(0.01)   # 1st percentile
upper = area_per_bathroom.quantile(0.99)   # 99th percentile
area_per_bathroom_clipped = area_per_bathroom.clip(lower, upper)

plt.figure(figsize=(10, 6))
sns.histplot(
    area_per_bathroom_clipped,
    bins=30,
    kde=True,
    color='salmon',
    edgecolor='black'
)

plt.xlabel('Area per Bathroom (m^2 / bathroom)')
plt.ylabel('Frequency')
plt.title('Distribution of Area per Bathroom', fontsize=14, weight='bold')
plt.grid(axis='y', alpha=0.75)
plt.show()
```



Integrity: Matches across systems or tables. An integrity analysis is not required, as all the data originates from a single source.

1.3 ETL Process

Following the completion of the data quality assessment, we will proceed to design and implement an automated ETL process, informed by the collected information, that will prepare the data for use in our interactive dashboard and Machine Learning layer. The process is documented in the file ETL-creation.ipynb.